

Disciplina:
**Laboratório de
Banco de Dados
Avançado**

Unidade 4:

Stored procedures

Definição, características e codificação de *stored procedures*.



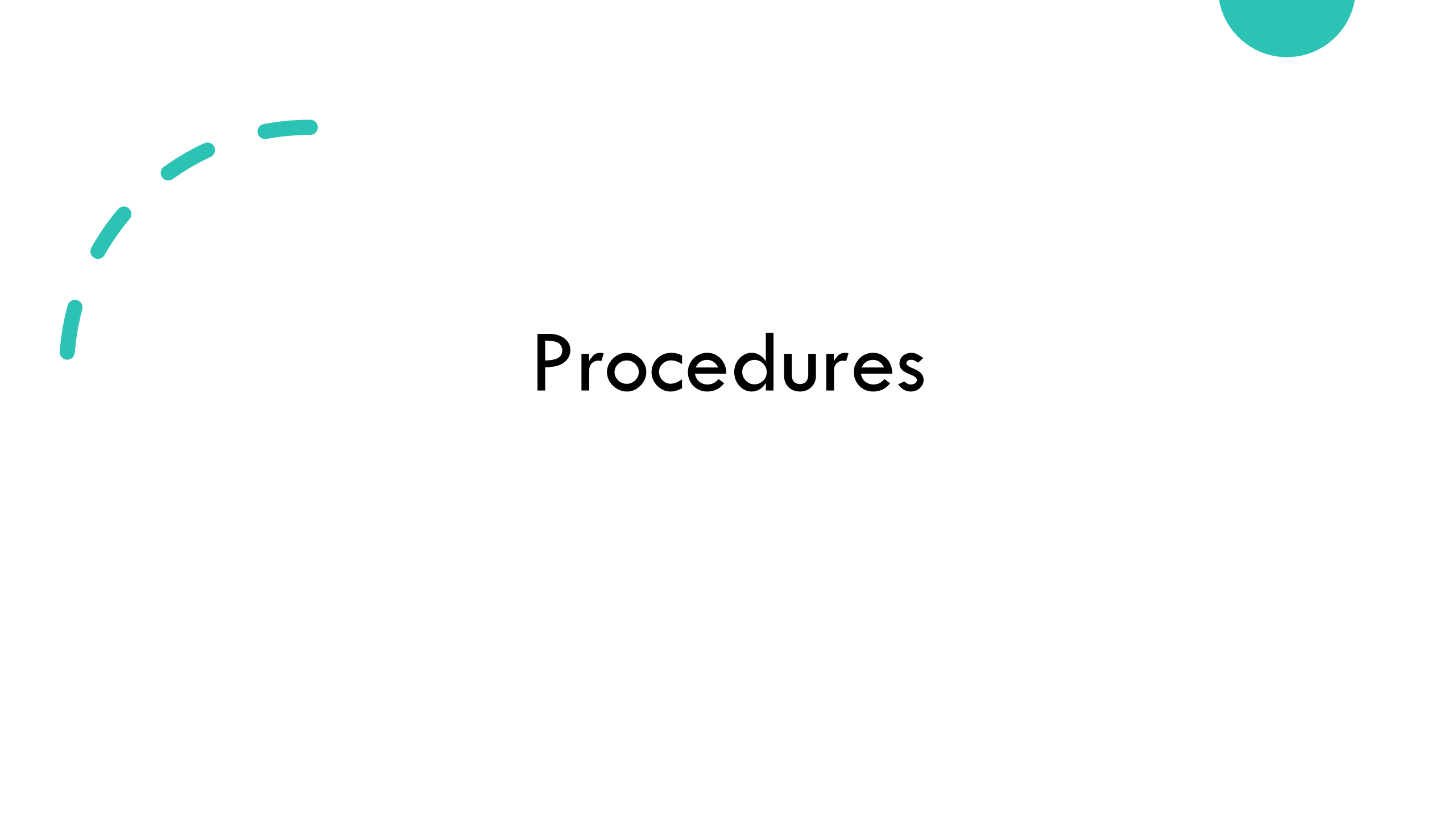
Algumas funções com varchar2

```
declare
    disciplina varchar2(40) := 'Laboratório de Banco de Dados Avançado';
    curso varchar2(40) := 'Ciência da Computação';
    tudo varchar2(80);
    email varchar2(40) := 'aluno@cruzeirosul.edu.br';
    texto varchar2(40) := 'Aula de Hoje';
begin

    dbms_output.put_line(length(disciplina));
    dbms_output.put_line(upper(disciplina));
    dbms_output.put_line(lower(disciplina));

    tudo := concat(disciplina, curso);
    dbms_output.put_line(tudo);

    dbms_output.put_line(substr(curso, 2, 7));
    dbms_output.put_line(instr(email, '@'));
    dbms_output.put_line(replace(disciplina, 'Banco de Dados', 'Database'));
    dbms_output.put_line(trim(texto));
end;
```



Procedures

Definição

- **Procedure:**

- São subprogramas que executam uma série de instruções PL/SQL. Podem ter parâmetros de entrada e saída.
- Podem ou não retornar um valor.

- **Function**

- Quando precisamos que um programa retorne uma informação.
- As funções sempre devem retornar um valor.
- Não pode ter em sua composição comandos DML, DDL e DCL, apenas *selects*.

Até o momento...

- Construímos programas em blocos chamados blocos anônimos.
- Caso quiséssemos guardá-los, teríamos que salvá-los em um ou mais arquivos em algum diretório no computador para não perdê-los.

Agora vamos guardar os programas no BD

- Vantagens:
- Reaproveitamento de códigos;
- Rapidez;
- Controle de alterações;
- Controle de acesso;
- Modularização.

Reaproveitamento de códigos

- Podemos escrever procedimentos e funções que podem servir como base para as demais partes de um sistema. Por exemplo, podemos criar uma função para validação de números de CPF que possa ser utilizada em vários módulos do sistema, como no módulo de RH ou de compras, ou seja, podemos ter um programa gravado no banco de dados ao qual todos podem ter acesso

Rapidez

- Com o programa armazenado no banco de dados, podemos acessá-lo rapidamente, sem ter que utilizar chamadas externas a arquivos. outro ponto interessante é que as ferramentas de desenvolvimento e análise do Oracle enxergam de forma nativa estes objetos armazenados.

Controle de alterações

- Um programa armazenado no banco de dados é muito mais fácil de ser alterado. É possível abri-lo e alterá-lo de forma mais rápida, compilando e em seguida efetivando as alterações. Como o programa está em um único lugar, os demais sistemas que o utilizam enxergarão todas as alterações realizadas.

Controle de acesso

- Por meio de concessões é possível limitar os acessos a estes programas, permitindo-os a apenas alguns usuários.

Modularização

- Pelo fato de estarem armazenados em um bd, os programas podem ser agrupados dentro de pacotes, o que permite que nós os organizemos e estructuremos de acordo com seus escopos, viabilizando a modularização do sistema.

Onde podemos executar os programas armazenados?

- Ferramentas SQL*Plus, Oracle Forms, Oracle Reports etc.
- Podem ser chamados por outros programas armazenados ou por blocos PL/SQL anônimos.

Exemplo de criação de uma procedure

```
1 v create procedure calc is
2   --
3   x1 number := 10;
4   x2 number := 5;
5   op varchar2(1) := '+';
6   res number;
7   --
8 v begin
9   if (x1 + x2) = 0 then
10    dbms_output.put_line('Resultado: 0');
11 v   elsif op = '*' then
12    res := x1 * x2;
13 v   elsif op = '/' then
14    if x2 = 0 then
15    dbms_output.put_line('Erro de divisão por zero!');
16 v   else
17    res := x1 / x2;
18   end if;
19 v   elsif op = '-' then
20    res := x1 - x2;
21 v   if res = 0 then
22    dbms_output.put_line('Resultado igual a zero!');
23 v   elsif res < 0 then
24    dbms_output.put_line('Resultado menor que zero!');
25 v   elsif res > 0 then
26    dbms_output.put_line('Resultado maior1 a zero!');
27   end if;
28 v   elsif op = '+' then
29    res := x1 + x2;
30 v   else
31    dbms_output.put_line('Operador inválido!');
32   end if;
33   dbms_output.put_line('Resultado do cálculo: '||res);
34 end;
```

Exemplo de execução da procedure

```
35  
36  -- executando a procedure  
37  exec calc;  
38  
39  execute calc;  
40  
41  begin  
42      calc;  
43  end;
```



Comando *replace* na criação de uma procedure

- Podemos utilizar o comando *replace* na criação de uma procedure.
 - `create or replace procedure calc is ...`
- 

Alteração de procedure

- Quando alteramos uma *procedure* ou *function* no banco de dados, pode acontecer de termos que compilá-la novamente. Para isso, utilizamos o comando *alter*.
- Exemplo:
- `alter procedure calc compile;`

Excluir uma procedure

- `drop procedure calc;`

Recuperando algumas informações

- Para visualizar informações referentes a procedures e functions utilize as views user_objects, all_objects ou dba_objects.

- select

- owner,
- object_type,
- status,
- created,
- last_ddl_time
 - from all_objects
 - where object_name = 'CALC';

OWNER	OBJECT_TYPE	STATUS	CREATED	LAST_DDL_TIME
SQL_NZXMBOFLDQWCRDQYBDHLHFKXR	PROCEDURE	VALID	25-SEP-24	25-SEP-24

- describe calc;

Argument Name	Type	In/Out	Default?
---------------	------	--------	----------

Recuperando algumas informações

- Para visualizar o código dos objetos armazenados no banco de dados, utilize as views user_source, all_source ou dba_source

- select
 - line,
 - text
 - from all_source
 - where
 - name = 'CALC';

LINE	TEXT
1	procedure calc is
2	--
3	x1 number := 10;
4	x2 number := 5;
5	op varchar2(1) := '+';
6	res number;
7	--
8	begin
9	if (x1 + x2) = 0 then
10	dbms_output.put_line('Resultado: 0');
11	elsif op = '*' then
12	res := x1 * x2;

Parâmetros nas Procedures

- Programas armazenados, como procedures e functions, permitem trabalhar com passagem de parâmetros.
- Quando criamos um procedimento ou uma função, podemos especificar parâmetros de entrada e saída para que valores possam ser levados para dentro destes programas ou recuperados deles.
- Os parâmetros podem ser do tipo in, out ou in out.
 - in define que se trata de um parâmetro de entrada.
 - Já um parâmetro do tipo out indica que ele é de saída.
 - Logo, in out indica um parâmetro de entrada e saída.

Parâmetros nas Procedures

- Quando estamos falando de parâmetros de entrada (in), isso quer dizer que os valores contidos neles podem ser atribuídos a outras variáveis ou parâmetros existentes dentro da procedure ou function. Contudo, não podemos atribuir valores a eles.
-
- Já quando estamos falando dos parâmetros de saída (out), estamos dizendo que os valores destes parâmetros podem ser alterados, mas não atribuídos a outras variáveis ou parâmetros existentes dentro destes objetos.
- Logo, quando temos parâmetros do tipo in out sendo usados, isso indica que tanto podemos atribuir valores a ele como também atribuir seus valores a outras variáveis ou parâmetros existentes dentro dos objetos.

Parâmetros nas Procedures

- Quando não informamos o tipo do parâmetro, por padrão, ele será do tipo in.

Exemplo de procedure com o uso de parâmetros

```
create or replace procedure somar(x1 in number, x2 in number, resp out number) is
begin
    resp := x1 + x2;
end;
```

```
declare
    exibir_resp varchar2(100);
begin
    somar(x1 => 3,
          x2 => 5,
          resp => exibir_resp);

    dbms_output.put_line('Total da soma: ' || exibir_resp);
end;
```

ou

```
declare
    exibir_resp varchar2(100);
begin
    somar(2,
          7,
          exibir_resp);

    dbms_output.put_line('Total da soma: ' || exibir_resp);
end;
```


Exemplo procedure com o uso de curso para listar dados

```
--Declaração da procedure para listar todos os alunos
create or replace procedure pr_listar_alunos is
    cursor aluno_cursor is
        select * from aluno;

    var_id aluno.idaluno%TYPE;
    var_nome aluno.nome%TYPE;
    var_nascimento aluno.data_nascimento%TYPE;
    var_email aluno.email%TYPE;
    var_telefone aluno.telefone%TYPE;

begin
    open aluno_cursor;
    loop
        FETCH aluno_cursor into var_id, var_nome, var_nascimento, var_email, var_telefone;
        exit when aluno_cursor%NOTFOUND;
        dbms_output.put_line('ID: ' || var_id ||
            ' Nome: ' || var_nome ||
            ' Nascimento: ' || var_nascimento ||
            ' E-mail: ' || var_email ||
            ' Telefone: ' || var_telefone
        );
    end loop;
    close aluno_cursor;
end;

--execução da procedure
exec pr_listar_alunos;
```

Exemplo de procedure com listagem de dados e parâmetro (sem o uso de cursor)

```
--procedure para listar os alunos por e-mail
create or replace procedure pr_listar_aluno_por_email(email_in in varchar2) is

    var_id aluno.idaluno%TYPE;
    var_nome aluno.nome%TYPE;
    var_nascimento aluno.data_nascimento%TYPE;
    var_email aluno.email%TYPE;
    var_telefone aluno.telefone%TYPE;

begin

    select * into var_id, var_nome, var_nascimento, var_email, var_telefone from aluno where email = email_in;

    dbms_output.put_line('ID: ' || var_id ||
        ' Nome: ' || var_nome ||
        ' Nascimento: ' || var_nascimento ||
        ' E-mail: ' || var_email ||
        ' Telefone: ' || var_telefone
    );

end;

exec pr_listar_aluno_por_email('carlos.silva@email.com');
```

Exemplo de procedure com o uso de exception

```
--procedure para listar os alunos por e-mail
create or replace procedure pr_listar_aluno_por_email(email_in in varchar2) is

    var_id aluno.idaluno%TYPE;
    var_nome aluno.nome%TYPE;
    var_nascimento aluno.data_nascimento%TYPE;
    var_email aluno.email%TYPE;
    var_telefone aluno.telefone%TYPE;

begin
    select * into var_id, var_nome, var_nascimento, var_email, var_telefone from aluno where email = email_in;

    dbms_output.put_line('ID: ' || var_id ||
                        ' Nome: ' || var_nome ||
                        ' Nascimento: ' || var_nascimento ||
                        ' E-mail: ' || var_email ||
                        ' Telefone: ' || var_telefone
                        );

exception
when NO_DATA_FOUND then
    dbms_output.put_line('Aluno não encontrado.');
```

```
end;

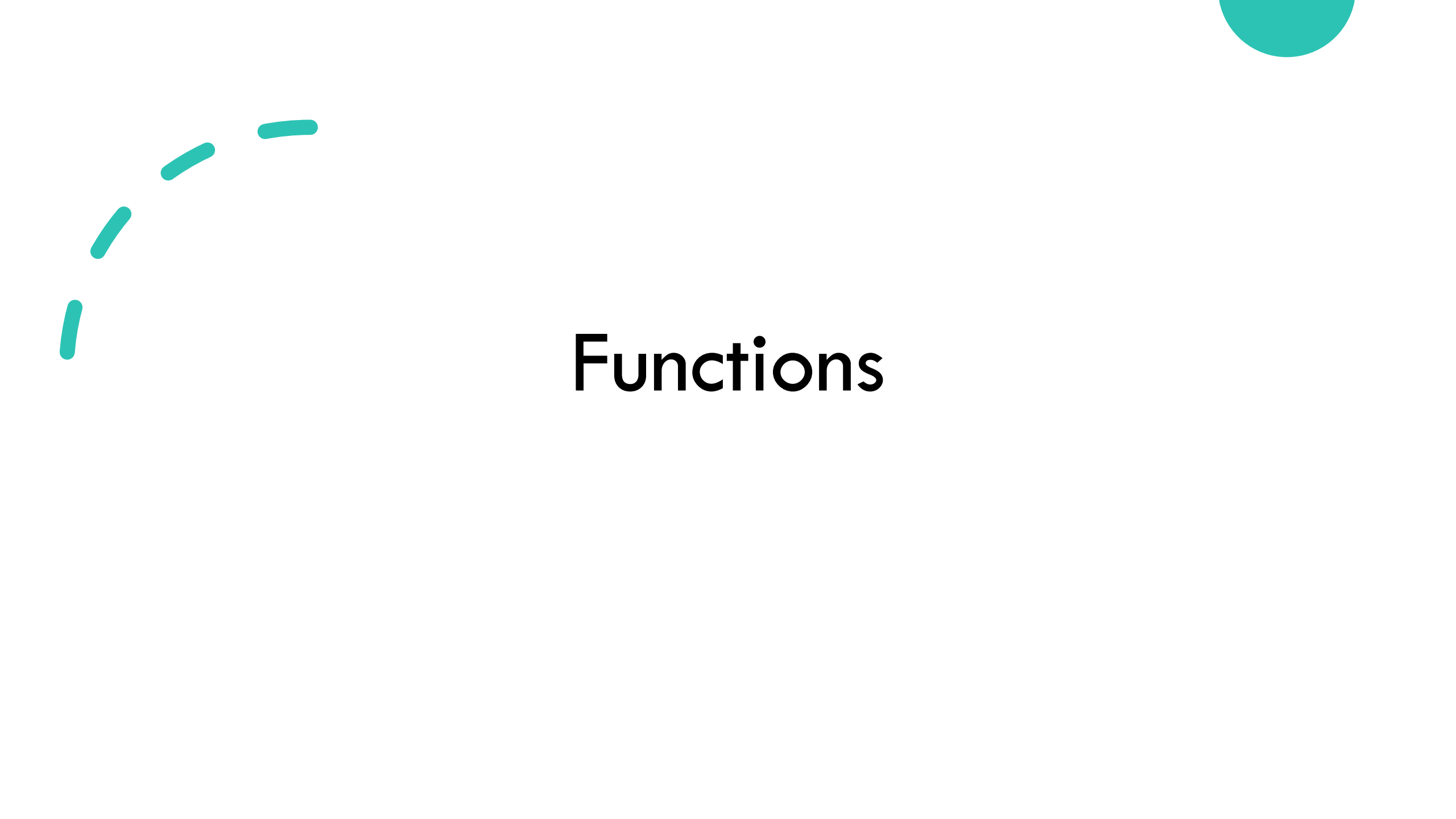
exec pr_listar_aluno_por_email('carlos.silva@email.com');
exec pr_listar_aluno_por_email('carlos2.silva@email.com');
```

Exercícios

1. Criar e executar uma procedure que faça uma inserção de aluno (a procedure deve verificar se o aluno já existe);
2. Criar e executar uma procedure que faça a alteração de um aluno de acordo com um idAluno informado (a procedure deve verificar se o idAluno informado existe);
3. Criar e executar uma procedure que exclua um aluno de acordo com um idAluno informado (a procedure deve verificar se o idAluno informado existe);

Exercícios

4. Criar e executar uma procedure que liste o nome do aluno, seu e-mail e sua turma;
5. Criar e executar uma procedure que exiba a quantidade de alunos de uma determinada turma;
6. Criar e executar uma procedure que exiba o id do último aluno inserido na tabela;
7. Criar e executar uma procedure que dado um cpf sem formatação, exiba o cpf formatado. Exemplo: 11122233396
CPF formatado: 111.222.333-96



Functions

Definição e exemplo

- Uma função é uma sub-rotina que aceita parâmetros, realiza um cálculo ou ação e retorna um valor. Exemplo:

```
1  -- Criação da function
2  v create or replace function soma(numero1 in number, numero2 in number)
3  return number is
4
5  begin
6      return numero1 + numero2;
7  end;
8
9  -- Execução da function
10 v begin
11 dbms_output.put_line('Total da soma:' ||soma(2,8));
12 end;
```

Mais um exemplo

```
create or replace function fatorial(numero in number)
```

```
return number is
```

```
fat number :=1;
```

```
begin
```

```
for contador in 1..numero loop
```

```
fat := fat * contador;
```

```
end loop;
```

```
return fat;
```

```
end;
```

```
begin
```

```
dbms_output.put_line(fatorial(5));
```

```
end;
```


Exemplo de *procedure* chamando uma *function*

```
create or replace function func_verifica_maioridade(numero in number)
    return boolean is
```

```
begin
    if(numero >= 18)then
        return TRUE;
    else
        return FALSE;
    end if;
end;
```

```
execute proc_exibir_dados('Ana',22);
```

```
create or replace procedure proc_exibir_dados(nome in varchar2, idade in number) is
    msg_maioridade varchar2(40);
```

```
begin
    if(func_verifica_maioridade(idade))then
        msg_maioridade := 'Maior de idade';
    else
        msg_maioridade := 'Menor de idade';
    end if;

    dbms_output.put_line('Nome: ' || nome || ' é ' || msg_maioridade);
end;
```

Exercícios

1. Crie uma função que receba o primeiro e o último nome de um cliente e retorne o nome completo.
2. Crie uma função que receba a data de nascimento de um cliente e retorne a idade.
3. Crie uma função que faça a inserção em uma tabela do banco de dados e retorne uma mensagem de sucesso ou erro.

Exercícios

- Criar uma function que verifique se um nome possui no mínimo 3 caracteres e retorne true caso verdadeiro ou false caso contrário. Criar outra function que verifique se um e-mail possui o caractere @ e mais de 8 caracteres. A função também deve retornar true ou false. Por fim crie e execute uma procedure que utilize as functions criadas para validar o nome e e-mail. Caso ambas functions retornem verdadeiro inserir os dados em uma tabela. Caso contrário exibir uma mensagem de erro ao cadastrar.

Contato

E-mail: allan@cruzeirosul.edu.br



Universidade **Cruzeiro do Sul**

www.cruzeirodosul.edu.br